Low-Bit Triton Kernels

Dr. Hicham Badri (@mobicham)
Principal Research Scientist
Mobius Labs GmbH

Nov 2nd 2024- 💎 GPU MODE Lecture

mobiuslabs

GPU MODE

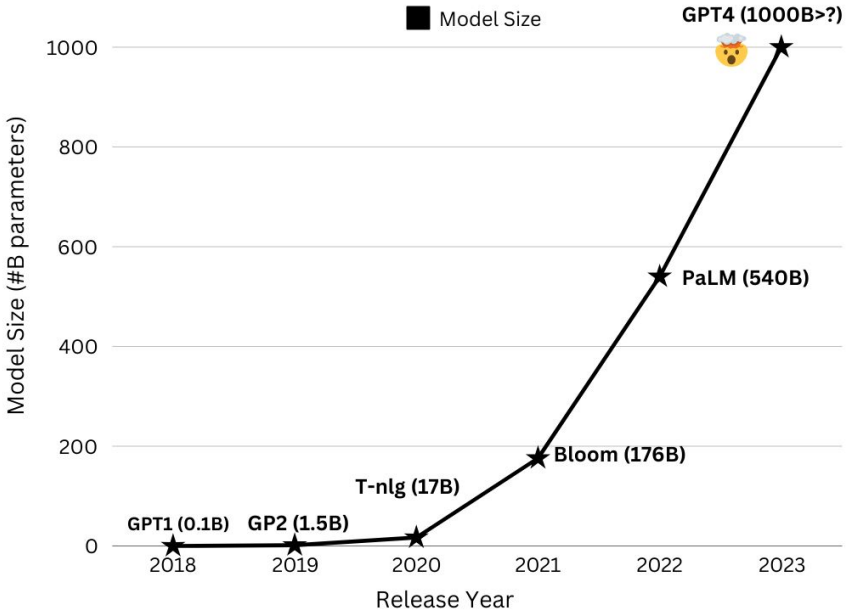
Overview

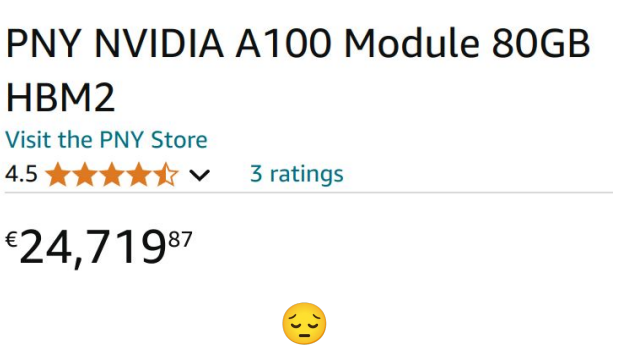
- **Introduction:** quantization, challenges and motivation to build GemLite 💎
- **Low-Bit Triton Kernels:** How to write various fast low-bit matmul Triton kernels 🧠
- **Integration:** How to integrate the kernels for faster runtime 🚀
- **Practical Example:** Running models with GemLite 🛠️
- **Future Work:** Current challenges and Santa wish list 🎅

Introduction

Introduction

- Machine learning models are getting **very large** (LLMs and VLMs).
- Running large models requires a lot of GPU **memory**:
  - Llama2-70B needs 140 GB of GPU memory, can't even fit in a single A100 80GB!
  - GPUs are very **expensive** (#GPU poor 😔).
  - Things are even worse for training …





Introduction

- In transformer-based models:
  - The large majority of the weights is in the **linear layers.**
  - Most of the compute is in the **linear layers** and the **attention** modules.

- Quantization is the key to run large transformer-based models efficiently:

  - Quantizing the linear layers **weights** results in:
    - Significant reduction in VRAM usage.
    - Significantly faster inference for memory-bound cases.

    ⇒ You can run a Llama 70B with **3.5x** less VRAM, **2.5-3x** faster inference, **8x** cheaper *without accuracy loss*!

  - Quantizing the **activations** results in:
    - Faster inference for compute-bound scenarios.

  - Quantizing the **KV cache** results in:
    - Significant reduction in VRAM usage for large context sizes.

  - **Transformers + quantization = less VRAM + faster!** 🚀

What is Quantization?

- Quantization is a technique to **reduce data bit precision:**
  - FP16/FP32 tensors can be quantized to INT8, FP8, INT4, FP4, INT2, ...

1.5	30	2.3	-1.3
-3	0.03	-4	20

x
FP16 tensor(16-bit)

x_quant = quantize_int4(x)

(15*(x - x.min())/(x.max() - x.min())).round()

2	15	3	1
0	2	0	11

x_quant
INT4 tensor (4-bit)

- We can quantize any large tensor:
  - Linear weights, activations in batched inference, KV cache for large context sizes, ...

- Challenges:
  - How to maintain **high-quality** outputs after quantization?
  - **Efficiency:**
    - Processing time - (both in the quantization and runtime steps).
    - The quantization step should ideally not require expensive hardware.

Weight and Activation Quantization Techniques

Weight Quantization:

- Linear / non-linear quantization:
  - Linear quantization is mathematically simpler, faster and typically efficient to apply (HQQ).
  - Non-linear quantization produces better results but tends to be slower for inference and can also be expensive at the quantization step (GPTVQ, VPTQ).

- Calibration-free / calibrated methods:
  - Calibrated methods require a calibration dataset, which could *overfit* to the data (GPTQ, AWQ).
  - Calibrated methods are much *slower* at the quantization step, but give better results.

Activation Quantization:

- Since we need to quantize on-the-fly, the quantization step needs to be very fast:
  - Typically INT8 / FP8 channel-wise symmetric quantization is enough
  - Some techniques like QuaRot quantize the activations to INT4.

We will focus on **linear quantization** for the rest of the presentation.

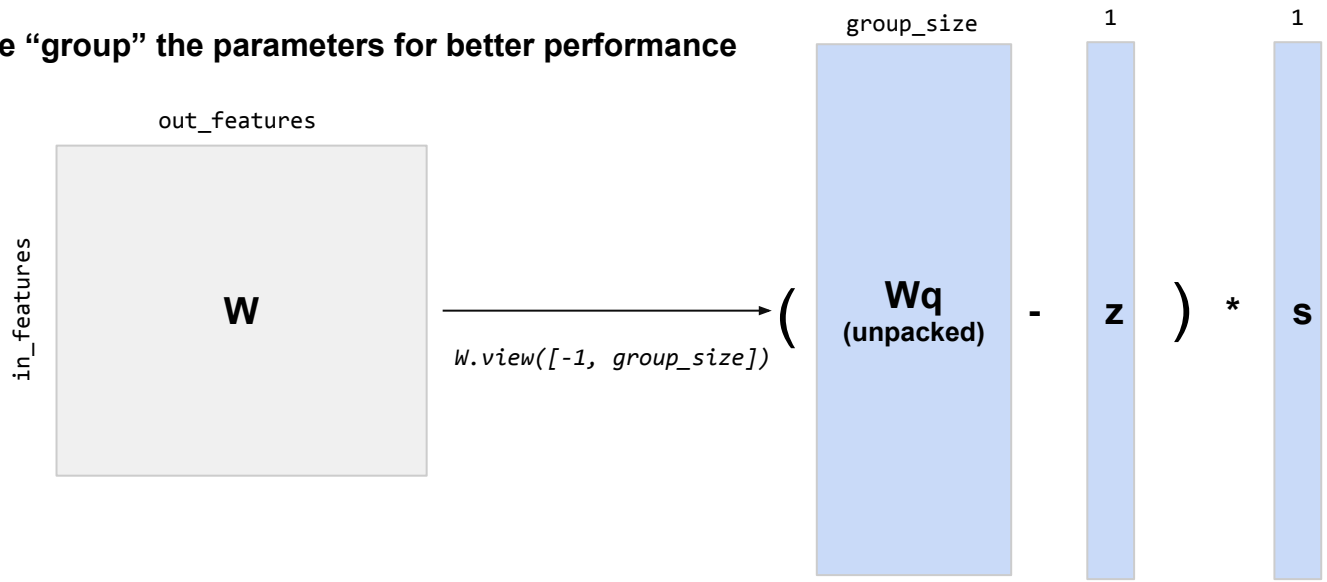
Linear Quantization: Overview

z=0 : symmetric quantization

$$Q_{z,s}(W) = \text{round}(W/s + z) = W_q$$ `Quantize(W)`

$$Q_{z,s}^{-1}(W_q) = s(W_q - z) = W_r$$ `Dequantize(Wq, z, s) ≈ W`

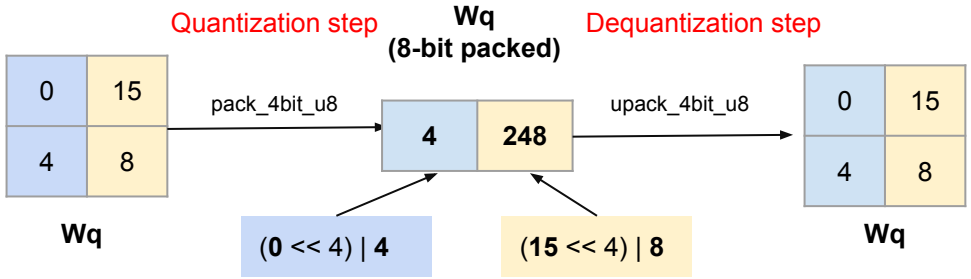
We "group" the parameters for better performance



Quantization methods like GPTQ, AWQ, HQQ **try to estimate the best (Wq, z, s) parameters!**

Linear Quantization: Bitpacking

- There's no native type (so far) for INT4, INT2, etc. in Pytorch, we need to "bitpack" the quantized tensors:
  - **4-bit**: bitpack 2x4-bit in 1 uint8,**or** 8x4-bit in 1 int32.
  - **3-bit**: bitpack 10x3-bit in 1 int32.
  - **2-bit**: bitpack 4x2-bit in 1 uint8 **or** 16x2-bit in 1 int32.



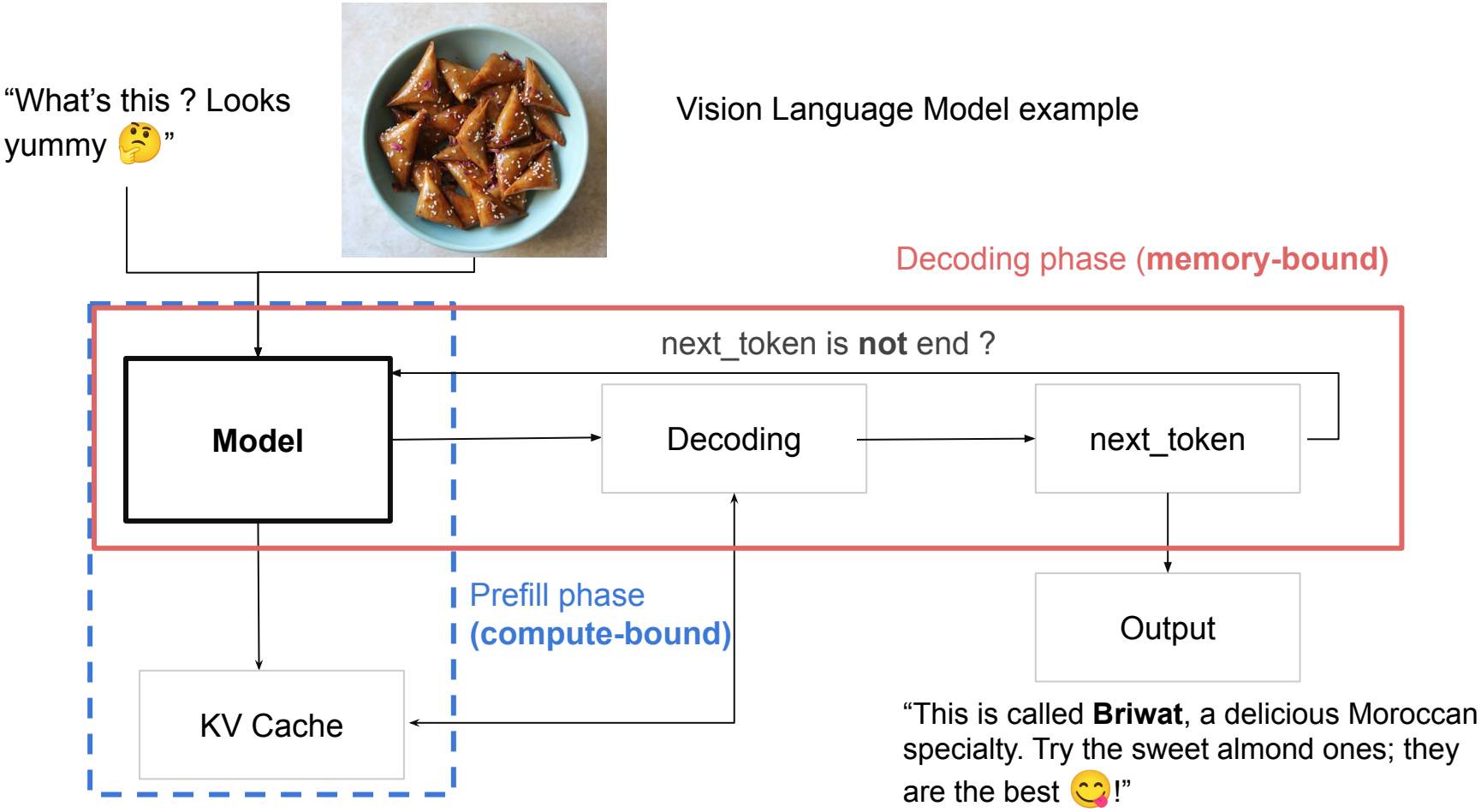

```python
def pack_4bit_u8(W_q): #uint8 > uint8/2
        W_q = W_q.to(torch.uint8)
        _step = int(len(W_q)/2)
        return (W_q[:_step] << 4) | W_q[_step:]
```

```python
def unpack_4bit_u8_cat(W_q): #uint8/2 > uint8
        return torch.cat([(W_q & 0b11110000) >> 4, W_q & 0b00001111], axis=0)
```

Now we want something like: **`matmul(x, W) ≈ matmul_lowbit(x, (bitpack(Wq), z, s))`**

BUT, how to perform **`matmul_lowbit`** efficiently? 🤔

Memory-Bound vs. Compute-Bound



Custom Kernels For Efficient "Low-Bit Matmul"

- Running **`matmul_lowbit`** requires a custom "fused kernel":
  - Running **`torch.matmul(x, dequantize(bitpack(Wq), z, s))`** is pretty slow due to the overhead to dequantize the *whole* weight matrix.
  - In order to benefit from the fast loading of the quantized weights, we need to "**fuse**" the weight loading with the dot product (more on that later)
  - Some open-source implementations are available, but they are very limited:
    - Fast custom CUDA kernels like *Tinygemm* (TorchAO) are only fast for batch-size>4
    - *Marlin* (VLLM) is the fastest, but only supports FP16 x INT4, with quantized zeros.
    - Solutions like *BitBlas* offer more dtypes but they are difficult to customize.
    - Some Triton kernels like Triton AutoGPTQ are easier to customize, but don't meet the desired performance.

We need to build a new easy to customize, high-performance, solution -> GemLite 💎

Why Triton?

- In order to achieve high-performance when deploying LLMs, we actually need to implement 3 versions of **`matmul_lowbit`** - more on that later.
- It's relatively easy to write a high-performance GEMV in CUDA, but not so easy to write a custom GEMM.
- Triton is much easier to write, debug and customize -> experiment and build faster!
- In many cases, Triton can achieve high performance, reaching ~90% of the speed of a highly optimized CUDA kernel created by an expert.
- Triton is relatively well integrated with Pytorch.
- Triton supports multiple devices (Nvidia gpus, AMD gpus, CPU)

Low-Bit Triton Kernels

Writing Low-Bit Matmul Kernels Challenges

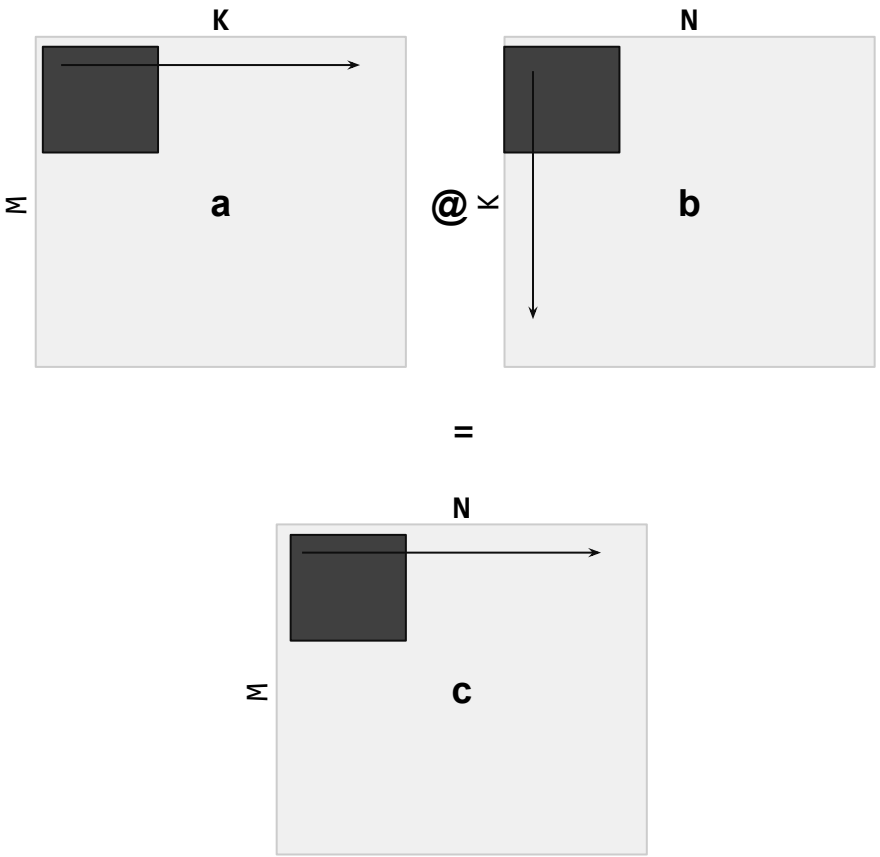
- **Multiple kernels are needed:**
  - We need a kernel for large prefills (GEMM)
  - We need a kernel for batched-decoding / short prefills (GEMM Split-K)
  - We need kernel for fast single user decoding (GEMV)

- **Same bitpacking across all kernels**:
  - The optimal bitpacking logic that would work for a GEMV is not necessarily the best one for other kernels like GEMM and GEMM Split-K
  - We need to use a flexible bitpacking logic and do the best with -> trade flexibility for a performance impact compared to a highly optimized specialized GEMV CUDA kernel.

- **Same kernel - multiple dtypes:**
  - We need to handle different input dtypes:
    - Activations can be FP16, INT8 or FP8
    - The weights can be packed Wn or unpacked like INT8 or FP8

Simple GEMM in Triton

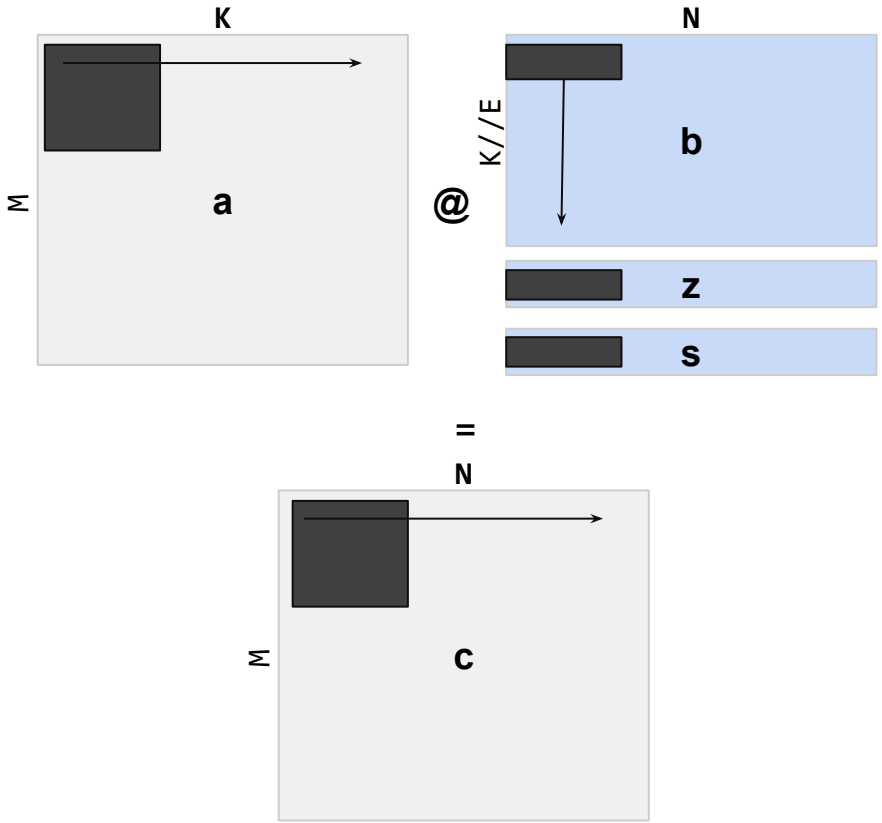
```python
pid = tl.program_id(axis=0)
num_pid_k = tl.cdiv(K, BLOCK_SIZE_K)

#Swizzle?
#pid_m, pid_n = linear_tile(pid, M, N, BLOCK_SIZE_M, BLOCK_SIZE_N, None)
pid_m, pid_n = swizzle_tile(pid, M, N, BLOCK_SIZE_M, BLOCK_SIZE_N, GROUP_SIZE_M)

#Offsets
offs_m  = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
offs_n  = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
offs_k  = tl.arange(0, BLOCK_SIZE_K)

#Inputs
a_ptrs  = a_ptr + (offs_m[:, None] * stride_am + offs_k[None, :] * stride_ak)
b_ptrs  = b_ptr + (offs_k[:, None] * stride_bk + offs_n[None, :] * stride_bn)

#Accumulator
acc = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=acc_dtype)

for k in range(num_pid_k):

    a = tl.load(a_ptrs)
    b = tl.load(b_ptrs)

    #Dot
    acc += tl.dot(a, b)

    #Advance
    a_ptrs += BLOCK_SIZE_K * stride_ak
    b_ptrs += BLOCK_SIZE_K  * stride_bk


#Output
c_ptrs  = c_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn)
tl.store(c_ptrs, acc)
```



Simple Low-Bit GEMM in Triton

```python
pid = tl.program_id(axis=0)
num_pid_k = tl.cdiv(K, BLOCK_SIZE_K)

#Swizzle?
#pid_m, pid_n = linear_tile(pid, M, N, BLOCK_SIZE_M, BLOCK_SIZE_N, None)
pid_m, pid_n = swizzle_tile(pid, M, N, BLOCK_SIZE_M, BLOCK_SIZE_N, GROUP_SIZE_M)

#Offsets
offs_m  = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
offs_n  = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
offs_k  = tl.arange(0, BLOCK_SIZE_K)

#Inputs
a_ptrs  = a_ptr + (offs_m[:, None] * stride_am + offs_k[None, :] * stride_ak)
b_ptrs  = b_ptr + ((offs_k[:, None] // elements_per_sample) * stride_bk + offs_n[None, :] * stride_bn)

unpack_mask: tl.constexpr = 0x0F #for 4-bit, 0x3 for 2-bit, etc.
q_shift = ((offs_k  % elements_per_sample) * W_nbits).to(tl.int32)[:, None]
s_ptrs  = scales_ptr + offs_bn[None, :] * stride_meta_n
z_ptrs  = zeros_ptr  + offs_bn[None, :] * stride_meta_n

#Accumulator
acc = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=acc_dtype)

for k in range(num_pid_k):

    a = tl.load(a_ptrs)
    b = tl.load(b_ptrs)

    #Load meta data
    k_m    = k * stride_mul
    zeros  = tl.load(z_ptrs + k_m * stride_meta_g)
    scales = tl.load(s_ptrs + k_m * stride_meta_g)

    #Unpack
    b = unpack(b, q_shift, unpack_mask)

    #dequantize
    b  = (b - zeros) * scales

    #Dot
    acc += tl.dot(a, b)

    #Advance
    a_ptrs += BLOCK_SIZE_K * stride_ak
    b_ptrs += (BLOCK_SIZE_K // elements_per_sample) stride_bk


#Output
c_ptrs  = c_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn)
tl.store(c_ptrs, acc)
```

Mainly used in the prefill phase (compute-bound)



Scales / Zeros

- We need to handle different use-cases:
  - “Grouped” scales and/or zeros -> these will be loaded **inside** the GEMM loop
  - “Channel-wise” scales -> these will be loaded **after** the GEMM loop
  - We need to handle activation scales as well:
    - If both the weights and the activations are channel-wise quantized, we can efficiently use the scales via an outer product: either via the scales of the weights if it applies, or via a vector of ones!

- We should *not* quantize the zeros to make the kernels compatible with methods like HQQ
- We should *not* pack the scales and zeros together since we need to support symmetric quantization

Dequantization Op

- The weights can be either:
  - Unpacked (bitwidth is 8-bit) -> we simply need to cast to the input dtype (FP16, INT8, FP8)
  - Packed (bitwidth < 8-bit) -> we need first to **unpack** the weights

- We need to load the packed weights with the same unpacked shape:
  - We a variable `elements_per_sample` that indicates how many values are packed per sample (e.g. 32 // 4 = 8 for 4-bit packed in 32-bit)
  - Currently the most efficient way to it is to repeat indices

```
b_ptrs  = b_ptr + ((offs_k[:, None] // elements_per_sample) * stride_bk + offs_bn[None, :] * stride_bn)
```

Currently this approach is unfortunately slower on some gpus like the A100 and H100

- After casting / unpacking, we need to **dequantize** ((Wq - z) * s) before performing the dot product.

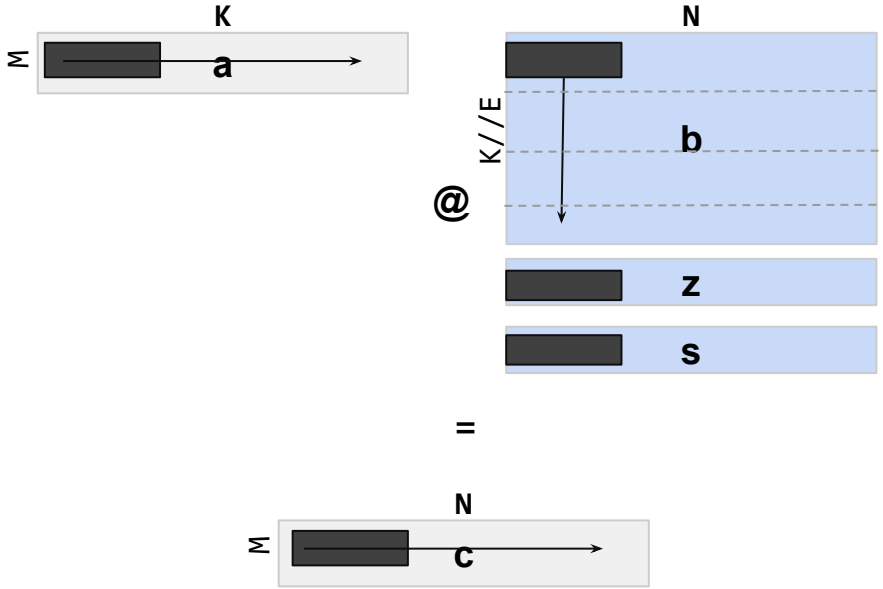
```
#Unpack
if(elements_per_sample > 1):
    q_shift = ((offs_k % elements_per_sample) * W_nbits).to(tl.int32)[:, None]
    b = (b >> q_shift) & unpack_mask #(int32)

#Dequantize
b = (b.to(target_dtype) - zeros) * scales #(target_dtype)
```

Simple Low-Bit GEMV in Triton

Mainly used in decoding phase with 1 query (memory-bound)

```python
#Swizzle?
pid_m, pid_n = linear_tile(pid, M, N, BLOCK_SIZE_M, BLOCK_SIZE_N, None)
#pid_m, pid_n = swizzle_tile(pid, M, N, BLOCK_SIZE_M, BLOCK_SIZE_N, GROUP_SIZE_M)

#Offsets
offs_m  = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
offs_n  = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
offs_k  = tl.arange(0, BLOCK_SIZE_K)

#Inputs
a_ptrs  = a_ptr + (offs_m[:, None] * stride_am + offs_k[None, :] * stride_ak)
b_ptrs  = b_ptr + ((offs_k[:, None] // elements_per_sample) * stride_bk + offs_n[None, :] * stride_bn)

unpack_mask: tl.constexpr = 0x0F #for 4-bit, 0x3 for 2-bit, etc.
q_shift = ((offs_k  % elements_per_sample) * W_nbits).to(tl.int32)[:, None]
s_ptrs  = scales_ptr + offs_bn[None, :] * stride_meta_n
z_ptrs  = zeros_ptr  + offs_bn[None, :] * stride_meta_n

#Load
a = tl.load(a_ptrs)
b = tl.load(b_ptrs)

#Load meta data
zeros  = tl.load(z_ptrs + k_m * stride_meta_g)
scales = tl.load(s_ptrs + k_m * stride_meta_g)

#Unpack
b = unpack(b, q_shift, unpack_mask)

#dequantize
b  = (b - zeros) * scales

#Dot product
acc = tl.sum(a.reshape((BLOCK_SIZE_K, 1), can_reorder=False) * b, axis=0, keep_dims=True)

#Output
c_ptrs  = c_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn)
tl.atomic_add(c_ptrs, acc, sem=atomic_mode)
```

No loop!

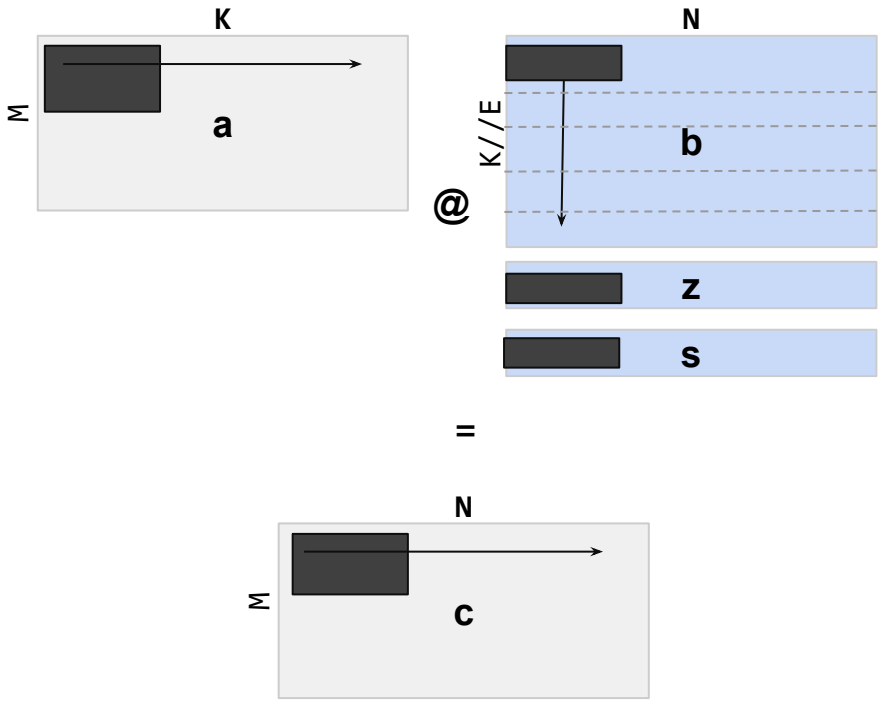


Don't forget to initialize the output with zeros!

Simple Low-Bit Split-K GEMM in Triton

```python
pid   = tl.program_id(axis=0)
pid_k = tl.program_id(axis=1)

#Swizzle?
#pid_m, pid_n = linear_tile(pid, M, N, BLOCK_SIZE_M, BLOCK_SIZE_N, None)
pid_m, pid_n = swizzle_tile(pid, M, N, BLOCK_SIZE_M, BLOCK_SIZE_N, GROUP_SIZE_M)

num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
num_pid_k = tl.cdiv(K, BLOCK_SIZE_K * SPLIT_K)

#Offsets
offs_m = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
offs_n = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
offs_k = pid_k * BLOCK_SIZE_K + tl.arange(0, BLOCK_SIZE_K)

#Inputs
a_ptrs  = a_ptr + (offs_m[:, None] * stride_am + offs_k[None, :] * stride_ak)
b_ptrs  = b_ptr + ((offs_k[:, None] // elements_per_sample) * stride_bk + offs_n[None, :] * stride_bn)

unpack_mask: tl.constexpr = 0x0F #for 4-bit, 0x3 for 2-bit, etc.
q_shift = ((offs_k  % elements_per_sample) * W_nbits).to(tl.int32)[:, None]
s_ptrs  = scales_ptr + offs_bn[None, :] * stride_meta_n
z_ptrs  = zeros_ptr  + offs_bn[None, :] * stride_meta_n

##############################################################################

acc = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=acc_dtype)

for k in range(num_pid_k):

    a = tl.load(a_ptrs)
    b = tl.load(b_ptrs)

    #Load meta data
    k_m    = (k * SPLIT_K + pid_k) * stride_mul #Needs pid_k!
    zeros  = tl.load(z_ptrs + k_m * stride_meta_g)
    scales = tl.load(s_ptrs + k_m * stride_meta_g)

    #Unpack
    b = unpack(b, q_shift, unpack_mask)

    #dequantize
    b  = (b - zeros) * scales

    #Dot
    acc += tl.dot(a, b)

    #Advance
    a_ptrs += BLOCK_SIZE_K   * SPLIT_K * stride_ak
    b_ptrs += (BLOCK_SIZE_K // elements_per_sample) * SPLIT_K * stride_bk

#Output
c_ptrs  = c_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn)
tl.atomic_add(c_ptrs, acc)
```

Mainly used in decoding phase with multiple queries(memory-bound)



Don't forget to initialize the output with zeros!

Triton Tricks!

- With some kernels, the loading order has a big impact on performance:
  - For example, loading the weights first before the activation can be faster than loading the activations first
  - It really depends on the kernel and the device !
  - One way to handle this is via autotuning: you put the model order a tunable parameter!

- Eviction policy can have a big impact on performance with small batches (GEMV/ Split-K):
  - We want to cache the activations via "`evict_last`", and give lower priority to the weights via "`evict_first`" (Inductor tends to generate kernels with similar eviction policies :))

```
a = tl.load(a_ptrs, mask=a_mask, others=0)
b = tl.load(b_ptrs)
```

→

```
if(A_load_order == 0): #Early load
    a = tl.load(a_ptrs, mask=a_mask, other=0., eviction_policy='evict_last')

b = tl.load(b_ptrs, eviction_policy='evict_first')

if(A_load_order == 1): #Late load
    a = tl.load(a_ptrs, mask=a_mask, other=0., eviction_policy='evict_last')
```

- On some GPUs, this for loop swap can avoid crashes and even run faster:

```
for k in range(num_pid_k):
    ...
```

→

```
for k in tl.range(0, num_pid_k, 1, num_stages=1):
    ...
```

More Triton Tricks!

- Since some configs in *autotune* will not be valid depending on the Split-K value and the group-size, we can automatically ignore them via:
  `prune_configs_by = {'early_config_prune': kernel_config_pruner}`

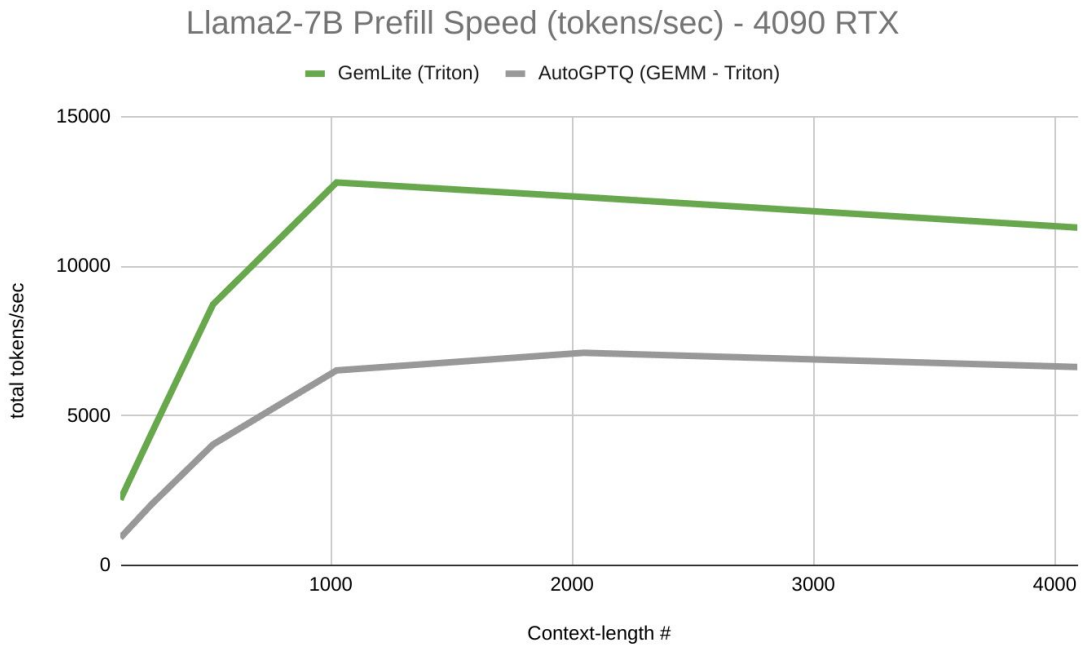
- A couple of <u>default parameters</u> are actually not the best:
  - `tl.atomic_atomic` tends to work the best with "`relaxed`" or "`release`" (the default is "`acq_rel`")
  - `reshape` works best with "`can_reorder=False`"

- For kernels that use atomic addition, it's faster to initialize the output via a <u>pre-hook</u>

```
def init_to_zero(name):
    return lambda nargs: nargs[name].zero_()

config = triton.Config(
    {"BLOCK_SIZE_M": 16, "BLOCK_SIZE_N": 64, "BLOCK_SIZE_K": 128},
    num_warps=4,
    num_stages=4,
    pre_hook=init_to_zero("c_ptr"),
)

def forward(...):
    output = torch.empty(...)
    kernel_call(...)
    return output
```

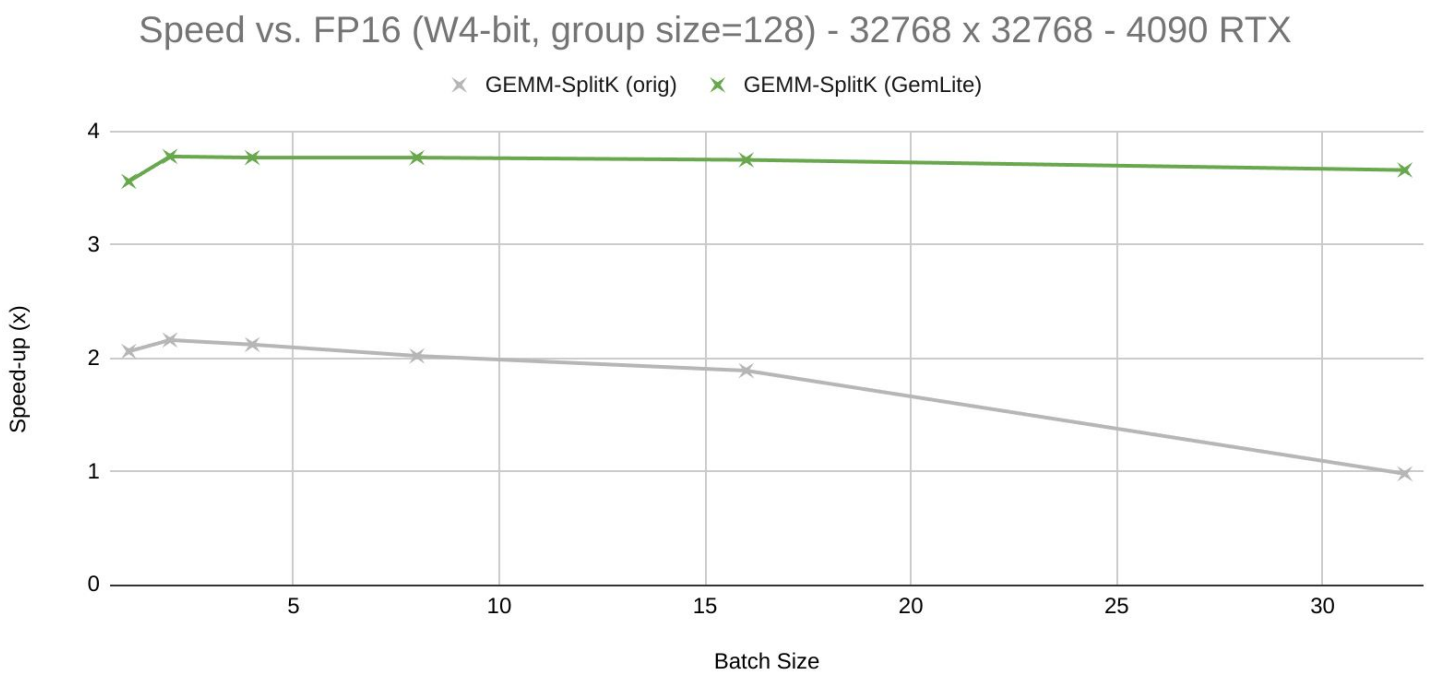
Triton Tricks Impact On Performance

Prefill Performance Comparison with AutoGPTQ Triton (GEMM)

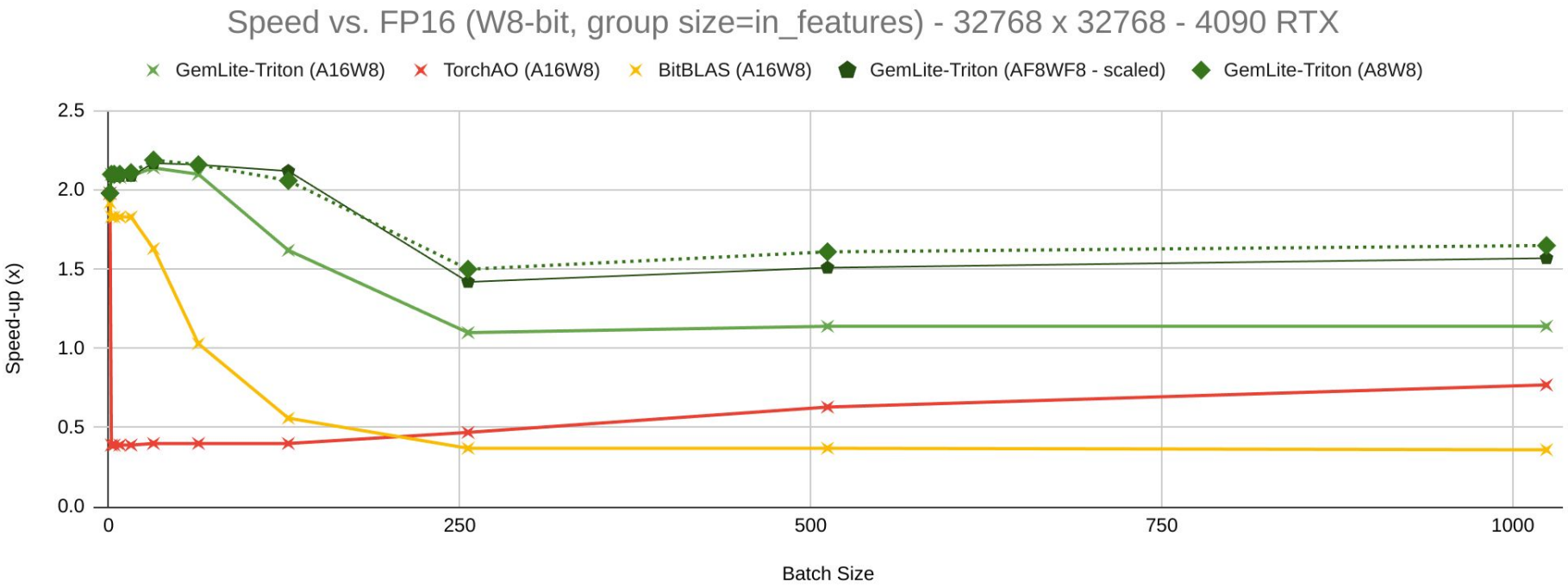


Triton Tricks Impact On Performance

GEMM Split-K Performance Comparison with
a reference Triton implementation

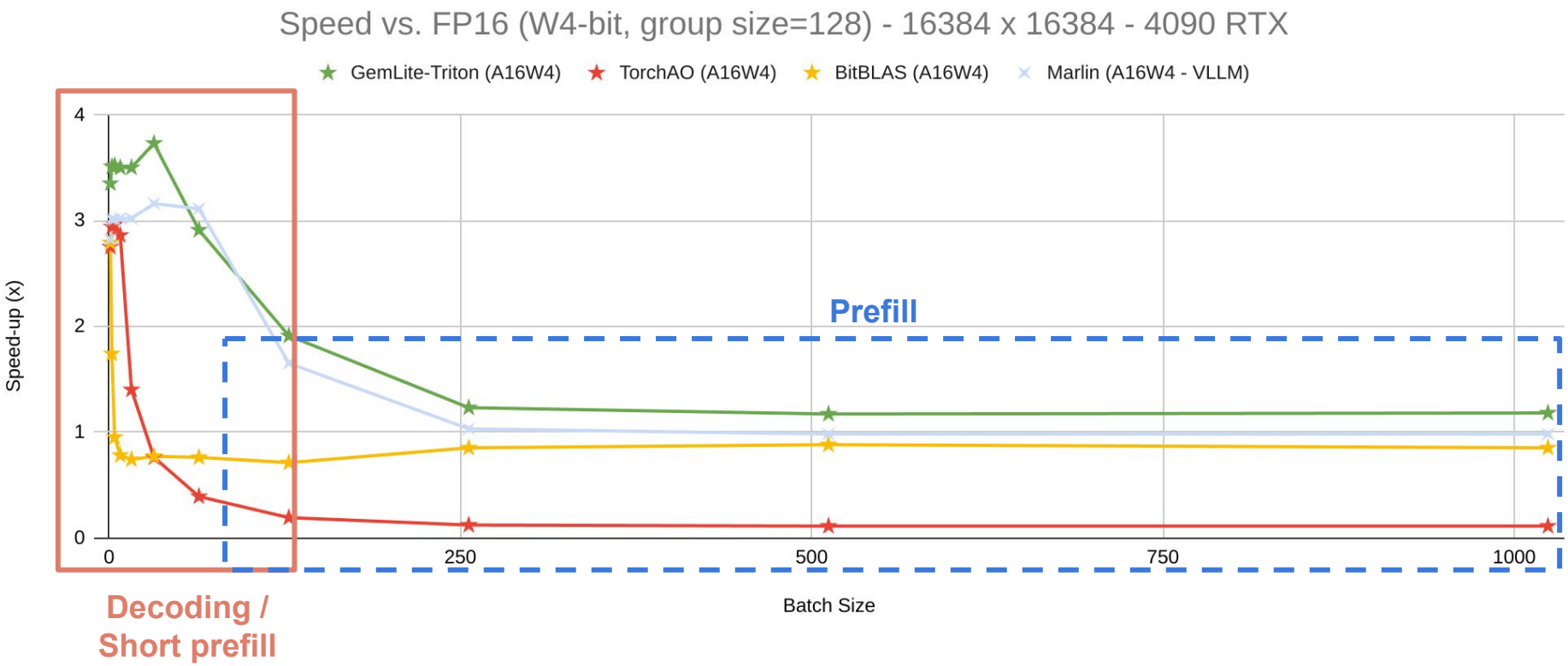


Matmul Benchmarks



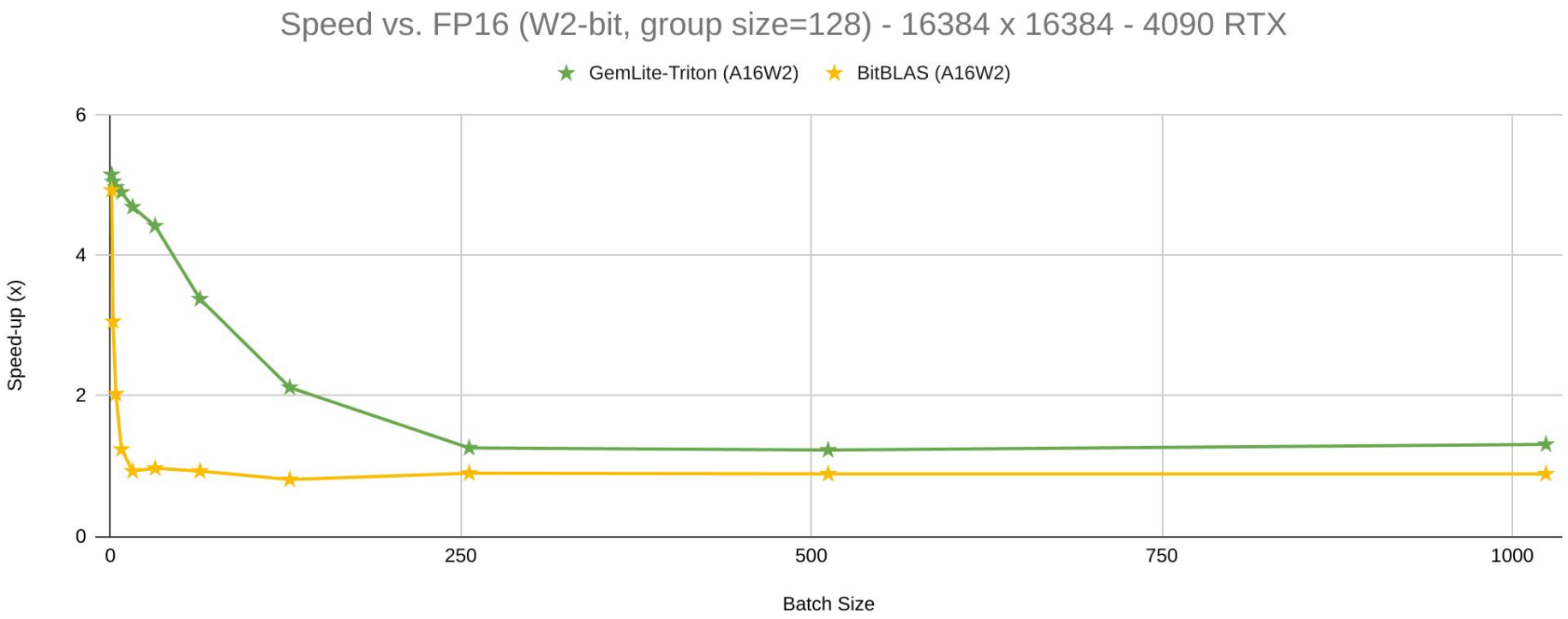
More results here

Matmul Benchmarks



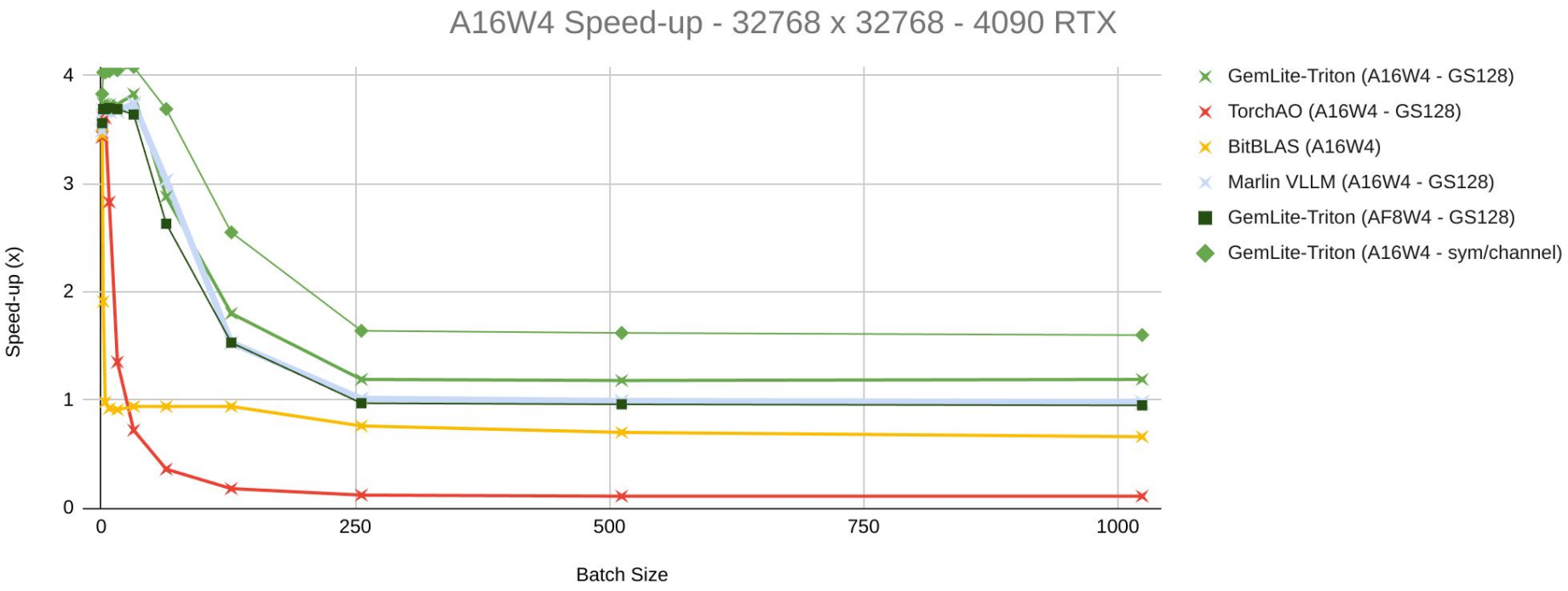
More results here

Matmul Benchmarks



More results here

Matmul Benchmarks



More results here

Supporting Odd Bitwidths

- Even bitwidths are easy to deal width, since each **BLOCK_SIZE_K** is divisible by the number of packed elements!
- How to support odd bitwidths like 5-bit and 3-bit?
  - Pack the quantized matrix into a set of two even bitwidth matrices via bit ops!

```
acc = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=acc_dtype)

for k in tl.range(0, num_pid_k, 1, num_stages=1):
    b1 = ((tl.load(b_ptrs_1, eviction_policy='evict_first') >> q_shifts1) & unpack_mask_1).to(tl.uint8) #(BLOCK_SIZE_K, BLOCK_SIZE_N)
    b2 = ((tl.load(b_ptrs_2, eviction_policy='evict_first') >> q_shifts2) & unpack_mask_2).to(tl.uint8) #(BLOCK_SIZE_K, BLOCK_SIZE_N)

    k_m    = (k * stride_mul).to(tl.int32)
    scales = tl.load(scales_ptrs + k_m * stride_meta)
    zeros  = tl.load(zeros_ptrs  + k_m * stride_meta)

    a = tl.load(a_ptrs, mask=a_mask, other=0., eviction_policy='evict_last') #(BLOCK_SIZE_M, BLOCK_SIZE_K)

    # Unpack and dequantize
    b = (((b2 << 1) | b1).to(a.dtype) - zeros) * scales

    #Dot
    acc = tl.dot(a, b, acc=acc, out_dtype=acc_dtype, input_precision="ieee") #(BLOCK_SIZE_M, BLOCK_SIZE_N)

    #Advance
    a_ptrs   += BLOCK_SIZE_K
    b_ptrs_1 += (BLOCK_SIZE_K // elements_per_sample_1) * stride_bk_1
    b_ptrs_2 += (BLOCK_SIZE_K // elements_per_sample_2) * stride_bk_2


tl.store(c_ptrs, acc, mask=(offs_am[:, None] < M) & (offs_bn[None, :] < N))
```

We load the 2x packed weights unpack them

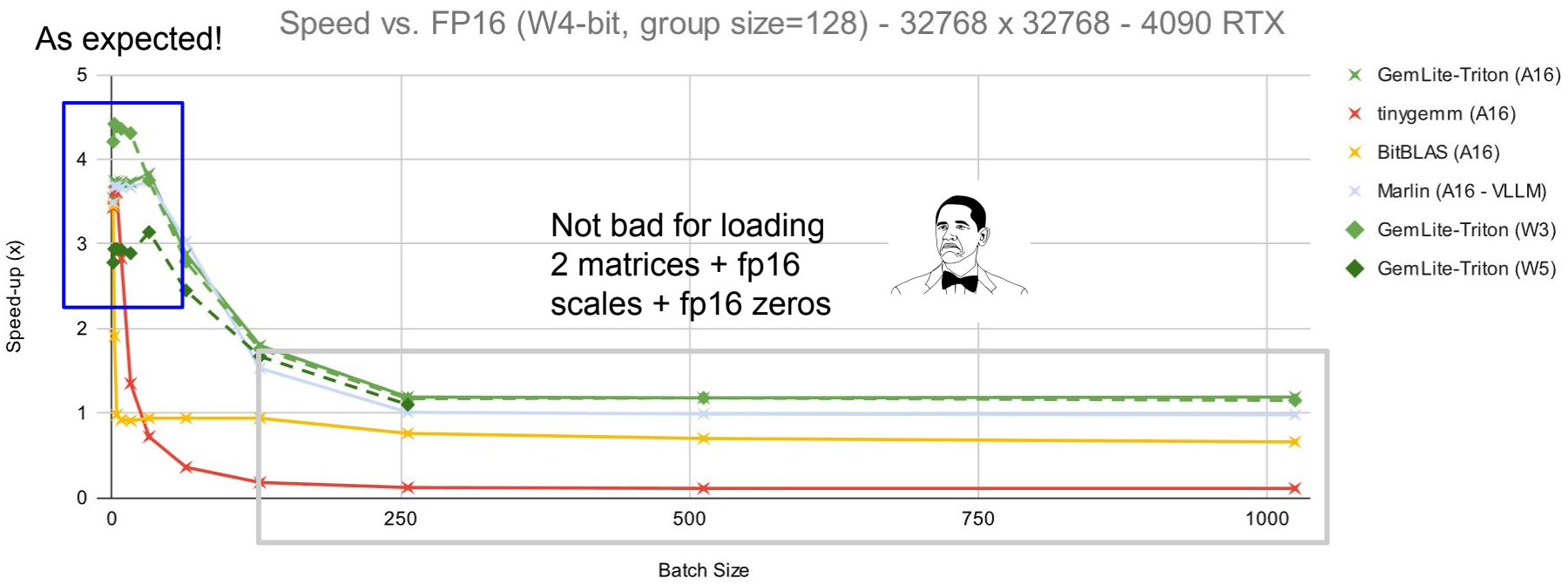
Combine the result then dequantize

Update

Supporting Odd Bitwidths

A16W3 GEMM performance with “double packing”

As expected!



Integration

Triton Kernel Integration

- Triton kernels need to be integrated end-2-end.
- We need to make sure tools like `torch.compile` and CUDA Graphs work with the kernels:
  - Check the input arguments to the kernel. Sometimes, unused arguments or some keywords like `num_stages` can cause `torch.compile` to break or silently output incorrect results.
  - Some important Triton features like pre-hooks and early config pruning are not supported by `torch.compile`:
    - Current solution is to register a `custom_op`
    - This also helps avoid some compilation issues (autotune-related)
    - However, this has a negative impact on the performance
- For CUDA Graphs:
  - Avoid using things like `.item()` which use the cpu device.
  - Sometimes, doing CUDA Graphs yourself can outperform `torch.compile` by a large margin 255 tokens/sec (`reduce-overhead`) -> 300 tokens/sec (`max-autotune-no-cudagraphs` + manual cuda graph)!

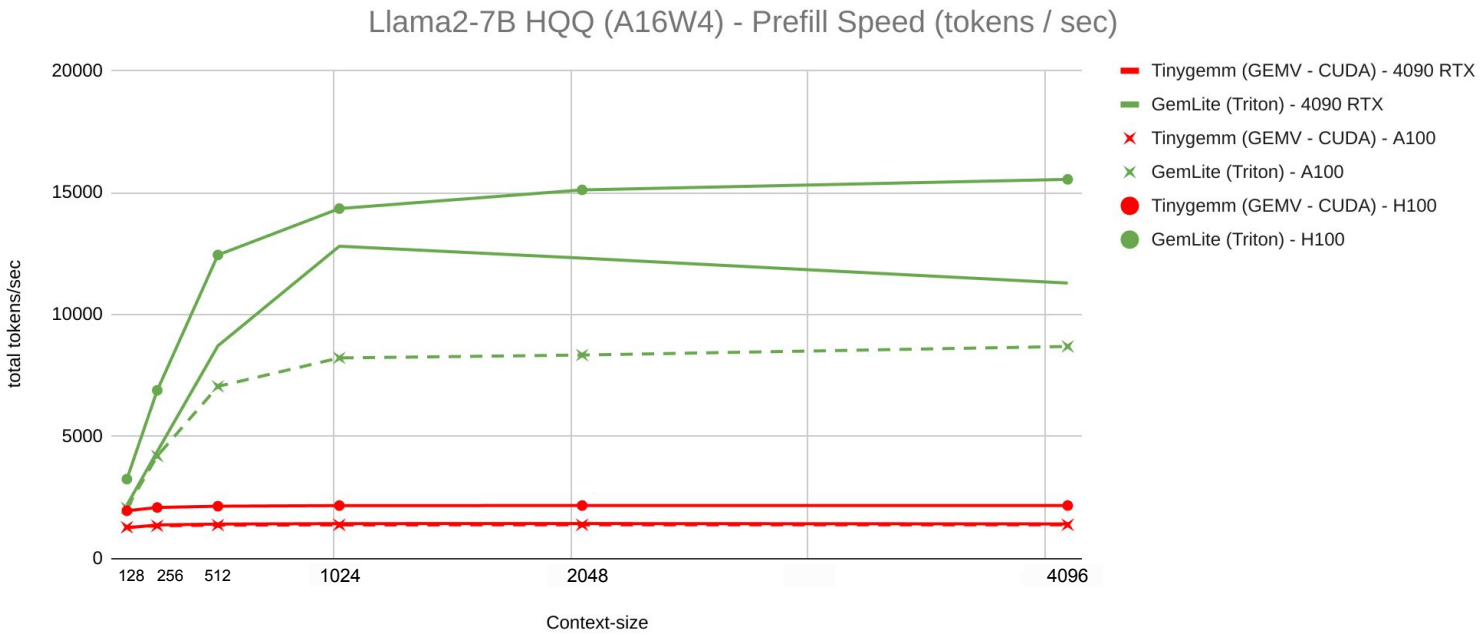
Triton Kernel Integration

- Low-bit kernels scheduling:
  - We want to use certain kernels in certain settings:
    - **batch-size = 1**: GEMV-variants + GEMM-SplitK -> Single user decoding.
    - **batch-size in [1, 32]**: GEMM Split-K -> Batched decoding / short prompt prefill.
    - **batch-size > 32**: GEMM -> Large batch decoding / medium to larger prompts.
  - The choice of which kernel is best for batch-size = 1 really depends on the matrix shape and the device -> run all of them and pick the best one ! (`exhaustive mode`)
- Reduce autotuning time:
  - Cache the exhaustive mode results for various shapes: only run once per device.
  - "Selective autotuning": run exhausting autotuning over various shapes and pick the most used values, across different devices!
  - Caching Triton autotune is not officially supported, but there are some tools like triton-dejavu

End-to-End Performance: Prefill Phase

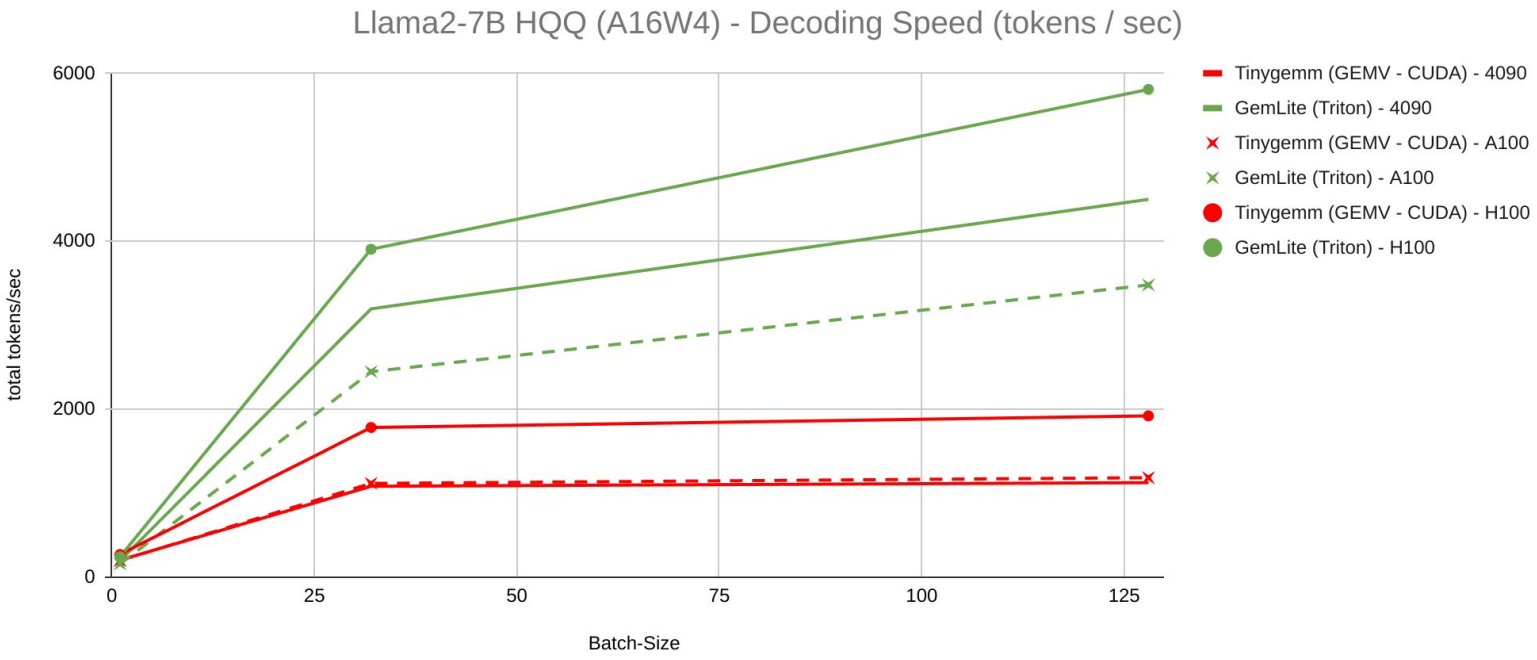
Llama2-7B (4-bit HQQ, gs=64) - Large Batch - (total tokens/sec - 🤗 model)
(work in progress)

- Up to 8x faster than the current ao solution!
- Up to 2x faster than the reference Triton GEMM implementation!



End-to-End Performance: Decoding Phase

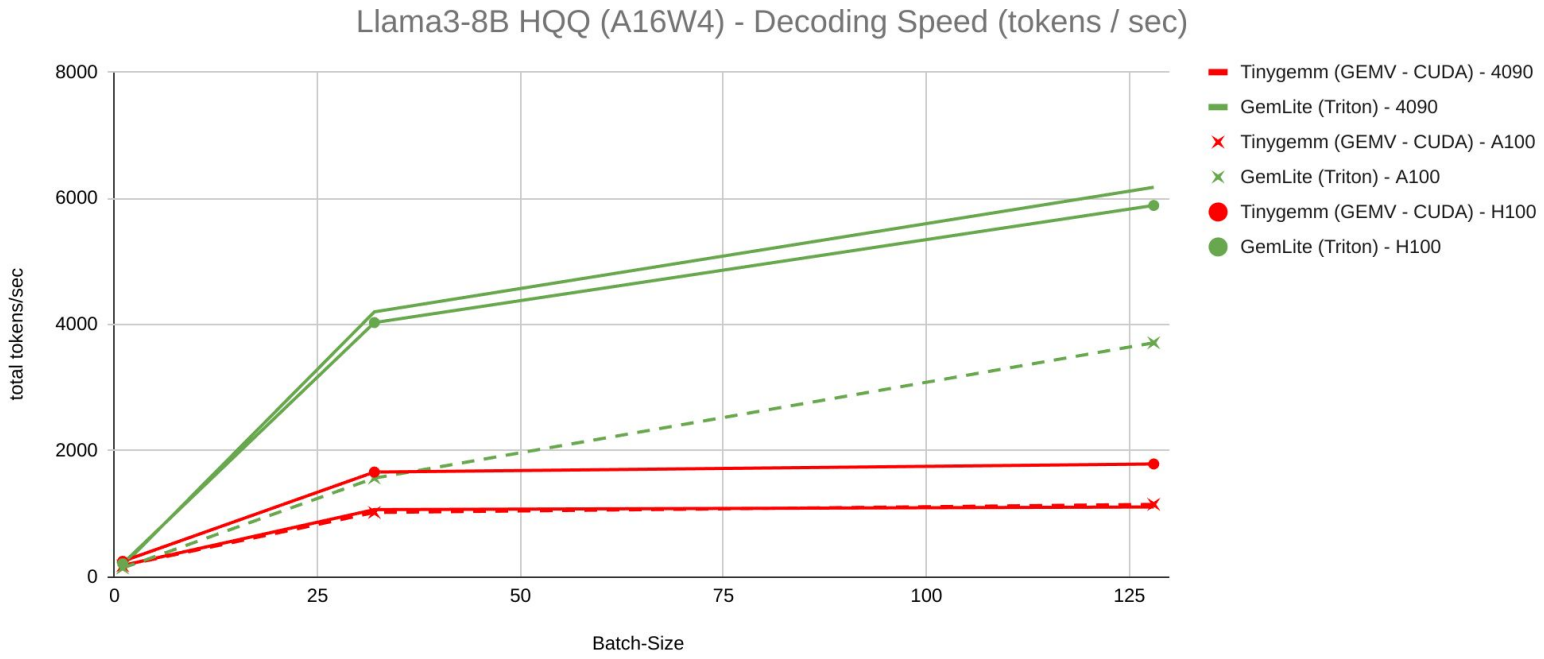
Llama (4-bit HQQ, gs=64) - Decoding (Small Batch) Speed - (total tokens/sec - gptfast)
(work in progress)



* thanks @HDCharles for the help with the benchmarks

End-to-End Performance: Decoding Phase

Llama (4-bit HQQ, gs=64) - Decoding (Small Batch) Speed - (total tokens/sec - gptfast)
(work in progress)



* thanks @HDCharles for the help with the benchmarks

Practical Example

Practical Example: HQQ + GemLite

- HQQ: for quantization + static/torch.compile runtime
- GemLite: for fused kernels
- TorchAO: integration in progress!

```
import torch
device        = 'cuda:0'
backend       = 'gemlite' #"torchao_int4", "bitblas", "gemlite"
compute_dtype = torch.bfloat16 if backend=="torchao_int4" else torch.float16
cache_dir     = '.'
model_id      = 'meta-llama/Llama-2-7b-chat-hf'

#Load model
from transformers import AutoModelForCausalLM, AutoTokenizer
from hqq.models.hf.base import AutoHQQHFModel
from hqq.core.quantize import *

#Load / Quantize
tokenizer = AutoTokenizer.from_pretrained(model_id, cache_dir=cache_dir)
model     = AutoModelForCausalLM.from_pretrained(model_id, cache_dir=cache_dir,
torch_dtype=compute_dtype, attn_implementation="sdpa")
quant_config  = BaseQuantizeConfig(nbits=4, group_size=128, axis=1)
AutoHQQHFModel.quantize_model(model, quant_config=quant_config, compute_dtype=compute_dtype,
device=device)

#Optimize
from hqq.utils.patching import prepare_for_inference
prepare_for_inference(model, backend=backend)

#Inference
from hqq.utils.generation_hf import HFGenerator
gen = HFGenerator(model, tokenizer, max_new_tokens=1000, do_sample=True, compile="partial").warmup()

#Generate
out = gen.generate("Write an essay about large language models.", print_tokens=False)
```

Llama2-7B 4-bit quantized requires ~5.5GB of VRAM

Practical Examples: HQQ + GemLite ~179 tokens/sec on a 4090 RTX = **8x speed-up**!

```
In [6]:     GEMLITE_TRITON_CACHE[signature] = {
                "forward": self.kernels[indx].forward,
In [6]:         "time": t[indx],
                "time_all": list(zip([k.matmul_type for k in self.kernels] , t))
In [6]:     }
In [6]:
In [6]:     def forward_auto(self, x):
In [6]:         if (len(GEMLITE_TRITON_CACHE) > 0):
                out_shape = x.shape[:-1] + (self.out_features,)
In [6]:         x_input = x.view(-1, x.shape[-1])
                args = [
In [6]:             x_input,
                    self.W_q,
In [6]:             self.scales,
                    self.zeros,
In [6]:             self.W_nbits,
                    self.group_size,
In [6]:             self.unpack_mask,
                    self.elements_per_sample,
In [6]:             self.acc_dtype,
                ]
In [6]:         _signature = (get_closest_m(x_input.shape[0]),) + self.signature
                if (_signature in GEMLITE_TRITON_CACHE):
In [6]:             self.warmup(_signature, args)
                out = GEMLITE_TRITON_CACHE[_signature]["forward"](*args).view(out_shape)
In [6]:
                if (self.bias != None):
In [6]:             out += self.bias
                return out
In [6]:     def forward_auto_no_warmup(self, x):
                if (x.view(-1, x.shape[-1]).shape[0] <= 16):
In [6]:             out = self.forward_manual(x, matmul_type='GEMM_SPLITK')
                else:
In [6]:             out = self.forward_manual(x, matmul_type='GEMM')
                return out
In [6]:
In [6]:     def forward_manual(self, x, matmul_type="GEMM"):
                out_shape = x.shape[:-1] + (self.out_features,)
In [6]:
In [6]:         out = (
                    GEMLITE_TRITON_MAPPING[(self.input_dtype.value, matmul_type)]
                    .forward(
In [6]: out = gen.generate("Write an essay about large language models.", print_tokens=True)
[ssh_tmux]0:screen*                                                    "root@C.13094651: ~" 13:02 11-Oct-24
```

Future Work

Future Work and Current Challenges

- Triton-lang:
  - Fix slow `tl.load` issue on some GPUs like the A100/H100 🎅
  - Fix the nightly build please 🎅
  - FP8 x FP8 with FP16 accumulation 🎅?
  - Extend to LUT-based quantization methods.
  - "Partial" TMA version for the GEMM kernel.
  - 3-bit / 5-bit integration.
  - Fused layers (MoEs, attention, etc.)
  - Extend to training (backprop version)
  - and much more!

Big thanks to the Pytorch AO and Inductor teams for the support!

- Torch / Inductor:
  - Better support for Triton kernels with `torch.compile` 🎅
    - `early_config_prune` support
    - `pre_hook` support
  - Better CUDA Graphs performance:
    - Manually doing CUDA Graphs outside of torch.compile can be significantly faster (?)
  - Autotuning not picking the best kernel ❗❓

Thank you for your attention 🫡!
Q&A